Experiment 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: Jashwanth Sundarajan | Register Number: 241401032

Aim

To study the limitations of low-code platforms when they are applied to complex enterprise-level applications and to understand when pro-code extensions or hybrid approaches may become necessary.

Introduction

Low-code platforms allow applications to be developed mainly through visual development tools instead of writing large amounts of traditional program code. Applications are assembled using predefined components such as forms, reports and dashboards, and behaviour is defined through workflows, rules and configuration. A small amount of scripting is usually available for specific needs. This approach reduces development time and effort and makes it possible for a wider range of people to take part in building business applications.

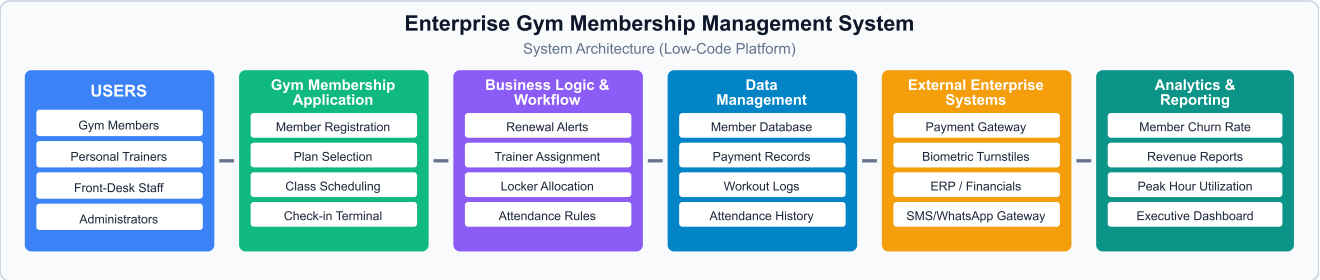
Low-code platforms are well suited to standard business applications. As the requirements of an enterprise grow in terms of users, data volume, process complexity, integrations, security and customisation, the built-in capabilities of a platform may no longer be sufficient on their own. This case study examines such limitations through an enterprise fitness chain scenario, which extends the Gym Membership Management application built in Zoho Creator.

Case Study: Enterprise Gym Membership Management System

A large multi-location fitness enterprise requires a system to manage its gym operations, starting from initial member registration and extending to trainer allocations, workout tracking, access control, recurring billing and franchise management. The system may need to serve the following users and handle the following functions:

Category	Items
Users	Prospective members, active members, personal trainers, front-desk staff, facility managers and system administrators
Data and operations	Member profiles, membership plans, trainer schedules, workout logs, diet plans, check-in records and locker assignments
Supporting functions	Automated billing, SMS/Email attendance alerts, payment processing, locker allocation and class booking
Enterprise integration and insight	Biometric access control integration, POS system integration, ERP/Finance integration, churn predictive analytics and executive dashboards

Figure 1 shows the architecture of this system on a low-code platform. The architecture is arranged as six groups that are connected from left to right:



Information flows from left to right: users → application → business logic → data → external systems → analytics

Figure 1: Enterprise Gym Membership Management System Architecture

The groups interact in sequence:

- **Users:** prospective members, active gym members, trainers, front-desk staff, facility managers and administrators, who interact with the system.
- **Gym Membership Application:** the user-facing part, covering member registration, membership plan selection, class booking, trainer assignment and check-in terminals.
- **Business Logic & Workflow:** renewal processing, automated billing rules, trainer availability checks, locker allocations and notifications, which drive operational flow.
- **Data Management:** the member database, payment records, workout logs and attendance history, in which information is securely stored.
- **External Enterprise Systems:** payment gateways, biometric turnstiles, ERP systems and SMS/WhatsApp services with which the application exchanges real-time data.
- **Analytics & Reporting:** churn predictive metrics, revenue reports, facility peak-hour utilization and management dashboards, which present key insights.

How a Low-Code Platform Can Support the System

A large part of the gym system consists of standard application functionality that low-code platforms are designed to handle efficiently:

Capability	Use in the Gym Membership System
Forms	Capturing member details, emergency contacts, medical history and selected plan via standard forms.
Reports	Viewing active members lists, plan expiration reports, daily attendance logs and trainer scheduling Kanban boards.
Workflows	Moving a member through registration, fee payment, active status, freeze status and renewal stages.
Business process automation	Triggering automatic confirmation emails or reminder SMS messages when a membership approaches expiry.
Data management	Storing member, trainer, equipment and payment records and executing standard CRUD operations.
User interfaces	Providing mobile-friendly web screens for member check-in, trainer session logging and admin setup.
Notifications	Alerting members regarding class schedule changes, payment due dates or facility maintenance.

Limitations at Enterprise Scale

As the gym management system expands across multiple branches with thousands of daily check-ins, the following enterprise limitations become critical. They are discussed individually below.

Performance and Scalability

During peak hours (early morning and evening), thousands of concurrent biometric check-ins across multiple branches hit the server simultaneously. Complex query execution for real-time occupancy counts, attendance tracking and live trainer schedule calculations can lead to latency. Low-code platforms manage the underlying infrastructure, restricting low-level database indexing or custom query optimization.

Complex Business Logic

Specialized logic—such as dynamic pro-rated membership plan upgrades, dynamic class cancellation penalty algorithms, multi-tier trainer commission structures, and automated locker reallocation matrices—is extremely cumbersome to configure using standard visual workflows or low-code drag-and-drop rule builders.

System Integration

Connecting the gym app to physical IoT hardware (biometric turnstile gates, smart lockers, RFID card scanners), Point-of-Sale (POS) registers for gym merchandise, and legacy financial ERP systems requires custom socket communication, webhook handlers and low-latency protocols that standard low-code connectors cannot handle out of the box.

Customization

Standard low-code UI templates are rigid. Building custom workout progress tracking graphs, interactive body-composition mapping visuals, branded multi-language member portals, or high-density trainer roster screens exceeds the visual rendering options provided by native low-code UI widgets.

Security and Governance

Gym management platforms store sensitive personal identifiable information (PII), biometric records, health conditions and payment details. Enterprises require strict RBAC (Role-Based Access Control), detailed audit logging, compliance with health data privacy regulations and enterprise SSO (Single Sign-On). Platform-provided security controls often lack fine-grained column-level access permissions necessary for high-compliance enterprise standards.

AI and Advanced Technology

Predicting member churn based on attendance drops, automated AI-driven personalized workout plan generation, and intelligent chatbot customer support require integrating custom machine-learning models (e.g., Python/TensorFlow services) which standard low-code platforms cannot natively host or execute.

Vendor / Platform Dependency

The enterprise becomes tied to the low-code vendor's pricing tiers, proprietary execution environment, and platform limits (API call quotas, storage limits). Migrating a mature multi-branch gym ecosystem off a proprietary low-code platform to a custom cloud infrastructure later involves complete re-engineering.

Data Management

Accumulating years of minute-by-minute biometric check-in data, sensor readings from smart fitness equipment, and financial transaction logs leads to massive data bloat. Standard low-code data layers struggle with heavy data transformations, cold-storage archival strategies and complex multi-table analytical joins.

Figure 2 summarizes how these limitations arise across the lifecycle of an expanding enterprise gym network.

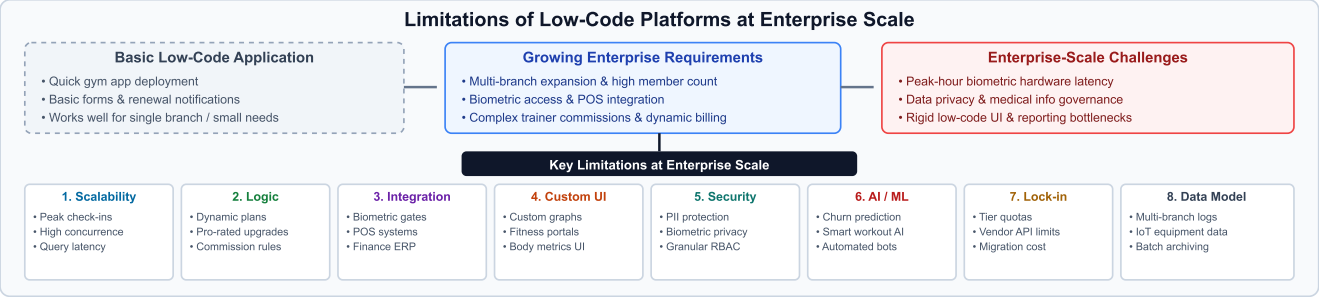


Figure 2: Limitations of Low-Code Platforms at Enterprise Scale

The relevance of each limitation to the Enterprise Gym Membership Management System is summarized below:

Limitation	Relevance to the Gym Case Study
Performance & Scalability	High volume of concurrent biometric check-ins during peak gym hours across multi-branch locations.
Complex Business Logic	Dynamic membership upgrades, class cancellation policies and multi-tier trainer commission tracking.
System Integration	Hardware integration with biometric turnstiles, smart lockers, POS counters and accounting software.
Customization	Interactive body composition progress dashboards and branded mobile applications for workout logging.
Security & Governance	Protecting health assessment records, payment instruments and biometric template data under privacy laws.
AI / Advanced Tech	Predicting member churn risk based on check-in frequency and AI-generated personal workout schedules.
Vendor Dependency	Reliance on platform execution speed, data throughput quotas and proprietary code constraints.
Data Management	Managing millions of check-in events, class bookings and sensor data streams across multi-year cycles.

Low-Code vs Pro-Code Requirements

The suitability of low-code or pro-code development depends on specific functional requirements within the gym system:

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard registration forms, admin grids and simple report views.	Interactive fitness progress charts, custom workout plan builders and branded member portals.
Business logic	Standard membership plan assignment and payment receipt workflows.	Dynamic pro-rated refund calculations, locker auto-allocation and trainer payout algorithms.
Data management	Built-in database storing profiles, plan types and payment histories.	High-throughput IoT sensor log storage and multi-branch data aggregation pipelines.
Integrations	Standard email notifications and basic credit card gateway links.	Low-latency biometric hardware APIs, RFID turnstiles and legacy financial ERP systems.
Automation	Scheduled birthday emails and automated plan expiration alerts.	Real-time gym capacity control actions and automated fraud check algorithms.
AI / ML	Basic platform dashboard analytics and simple static rule triggers.	Custom churn prediction models and automated AI diet/workout generator APIs.
Security	Standard role-based view permissions (Admin, Staff, Member).	Biometric hash encryption, HIPAA/GDPR health data compliance and column-level encryption.
Scalability	Sufficient for single-branch or mid-sized facility member volumes.	High-concurrency peak hour check-ins across national gym chains.

Hybrid Approach

Organisations can combine low-code development with pro-code extensions. Low-code is used to build standard front-desk features rapidly, while custom code, APIs and specialized microservices handle complex requirements.

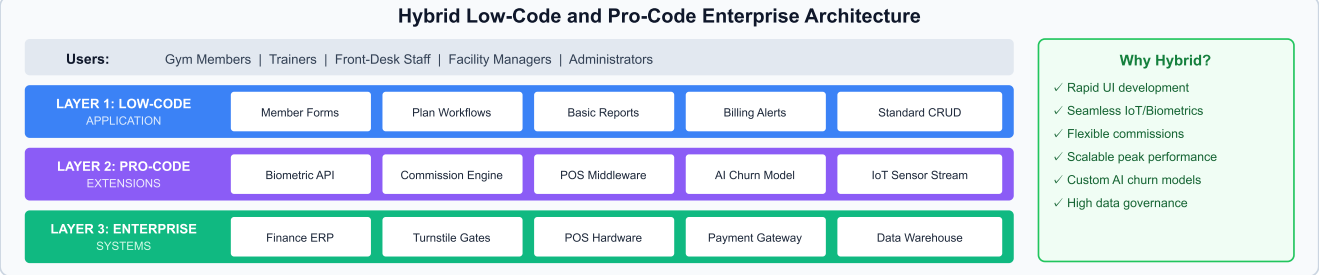


Figure 3: Hybrid Low-Code and Pro-Code Enterprise Architecture

The three layers of the hybrid architecture operate as follows:

Layer	Role and Functions
Layer 1 - Low-Code Application	Provides rapid development for standard operations: member intake forms, plan renewal views, payment alerts, staff scheduling dashboards and basic CRUD operations.
Layer 2 - Pro-Code Extensions	Adds high-performance logic: biometric turnstile REST APIs, dynamic trainer commission algorithms, POS middleware sync, AI churn prediction models and real-time attendance streams.
Layer 3 - Enterprise Systems	Core operational systems: enterprise finance ERP, physical turnstile hardware controllers, POS counters, gateway payment processors and analytical data warehouses.

When Pro-Code Extension May Become Necessary

- **Biometric access hardware integration:** when physical turnstiles require sub-second TCP socket or web-socket authentication protocols.
- **Complex commission algorithms:** when trainer payouts depend on variable class attendance thresholds, member ratings, and multi-tier plans.
- **Custom mobile user interfaces:** when members require smooth interactive workout logs and body measurement charting.
- **AI-driven member retention:** when machine-learning models predict churn risks based on historical check-in habits.
- **High concurrency during peak hours:** when thousands of members check in across branches within brief morning/evening windows.
- **Strict regulatory health data compliance:** when storing medical history or biometric hashes requires advanced custom field encryption.

Case Study Analysis

For the Enterprise Gym Membership Management System, a low-code platform like Zoho Creator is highly effective for rapidly building standard operational modules: member registration forms, basic membership renewal workflows, static class schedules, staff role permissions and standard revenue reports. These core capabilities were successfully demonstrated in the initial Gym Membership application.

However, as the gym enterprise scales into a multi-branch network, high concurrent biometric turnstile access, dynamic pro-rated plan logic, hardware POS integration, and predictive churn analytics introduce severe platform bottlenecks. Applying a hybrid approach allows the organization to preserve the rapid deployment speed of low-code for admin management while offloading hardware communication, complex billing rules and real-time analytics to pro-code microservices.

Result

The case study demonstrated the major limitations that arise when low-code platforms are applied to complex enterprise fitness networks: performance under peak check-in loads, complex commission logic, hardware integration, customized member UI, security/privacy governance, AI capability limits, vendor lock-in, and large-scale data stream management. It confirmed the effectiveness of a hybrid low-code/pro-code model.

Conclusion

Low-code platforms provide rapid development for standard business workflows, but their platform restrictions